Valdir de Souza Carvalho Neto **SI 241 - Algoritmos e Estruturas de Dados I**

Atividade 06

EXERCÍCIO 01

```c
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

#define MaxTam 5
typedef int Apontador;
typedef struct
{
    int cod;
    int idade;
} TipoItem;

typedef struct
{
    TipoItem Item[MaxTam];
    Apontador Topo;
} TipoPilha;

TipoPilha Pilha;
TipoPilha Pilha2;
TipoPilha Pilha3;
TipoItem x;

void FPVazia(TipoPilha *Pilha)
{
    Pilha->Topo = 0;
}

int Vazia(TipoPilha Pilha)
{
    return (Pilha.Topo == 0);
}

void Empilhar(TipoPilha *Pilha, TipoItem *x)
{
    if (Pilha->Topo >= MaxTam)
        printf("ERRO: Pilha está cheia");
```

```
    else
    {
        Pilha->Item[Pilha->Topo] = *x;
        Pilha->Topo++;
    }
}

void Desempilhar(TipoPilha *Pilha, TipoItem *x)
{
    if (Vazia(*Pilha))
        printf("ERRO: Pilha está vazia");
    else
    {
        *x = Pilha->Item[Pilha->Topo - 1];
        printf("O elemento retirado foi o de código %d\n", x->cod);
        Pilha->Topo--;
    }
}

void ImprimeFundoTopo(TipoPilha Pilha)
{
    if (Vazia(Pilha))
        printf("ERRO: Pilha está vazia");
    else
    {
        printf("\n");
        for (int i = 0; i < Pilha.Topo; i++)
        {
            printf("Cód: %d - Idade: %d\n\n", Pilha.Item[i].cod,
Pilha.Item[i].idade);
        }
    }
}

void ImprimeTopoFundo(TipoPilha Pilha)
{
    if (Vazia(Pilha))
        printf("ERRO: Pilha está vazia");
    else
    {
        printf("\n");
        for (int i = Pilha.Topo - 1; i >= 0; i--)
```

```
        {
            printf("Cód: %d - Idade: %d\n\n", Pilha.Item[i].cod,
Pilha.Item[i].idade);
        }
    }
}

int Tamanho(TipoPilha *Pilha)
{
    return Pilha->Topo;
}

void Procura(TipoPilha *Pilha, TipoItem *x)
{
    if (Vazia(*Pilha))
        printf("ERRO: Pilha está vazia");
    else
    {
        for (int i = Pilha->Topo - 1; i >= 0; i--)
        {
            if (Pilha->Item[i].cod == x->cod)
            {
                printf("\nElemento encontrado!\n");
                return;
            }
        }
        printf("Elemento não encontrado!\n");
    }
}

void Divide(TipoPilha *Pilha1, TipoPilha *Pilha2, TipoPilha *Pilha3)
{
    int aux = Pilha1->Topo / 2;
    if (Vazia(*Pilha1))
        printf("ERRO: Pilha está vazia");
    else
    {
        for (int i = Pilha1->Topo - 1; i >= aux; i--)
        {
            Pilha2->Item[Pilha2->Topo] = Pilha1->Item[i];
            Pilha2->Topo++;
            Pilha1->Topo--;
```

```
        }
        for (int i = Pilha1->Topo - 1; i >= 0; i--)
        {
            Pilha3->Item[Pilha3->Topo] = Pilha1->Item[i];
            Pilha3->Topo++;
            Pilha1->Topo--;
        }
        printf("\nPilha 2:\n");
        ImprimeTopoFundo(*Pilha2);
        printf("Pilha 3:\n");
        ImprimeTopoFundo(*Pilha3);
    }
}

void CalculaIdade(TipoPilha *Pilha)
{
    if (Vazia(*Pilha))
        printf("ERRO: Pilha está vazia");
    else
    {
        float media;
        int menor = Pilha->Item[Pilha->Topo - 1].idade, maior =
Pilha->Item[Pilha->Topo - 1].idade;
        for (int i = Pilha->Topo - 1; i >= 0; i--)
        {
            media += Pilha->Item[i].idade;
            if (Pilha->Item[i].idade < menor)
                menor = Pilha->Item[i].idade;
            if (Pilha->Item[i].idade > maior)
                maior = Pilha->Item[i].idade;
        }
        media /= Pilha->Topo;
        printf("A maior idade é %d\n", maior);
        printf("A menor idade é %d\n", menor);
        printf("A média das idades é %.2f\n", media);
    }
}

int main()
{
    int opcao, resposta;
    SetConsoleOutputCP(CP_UTF8);
```

```
    FPVazia(&Pilha);
    FPVazia(&Pilha2);
    FPVazia(&Pilha3);
    do
    {
        printf("\n\nMENU\n\n");
        printf("Digite a opção desejada:\n");
        printf("0 - Sair\n");
        printf("1 - Esvaziar Pilha\n");
        printf("2 - Verificar se a pilha está vazia\n");
        printf("3 - Inserir elemento na pilha\n");
        printf("4 - Retirar elemento da pilha\n");
        printf("5 - Imprimir os elementos da pilha no sentido
Fundo_Topo\n");
        printf("6 - Imprimir os elementos da pilha no sentido
Topo_Fundo\n");
        printf("7 - Verificar tamanho da pilha\n");
        printf("8 - Procurar elemento pelo código\n");
        printf("9 - Partir a pilha em duas pilhas\n");
        printf("10 - Calcular a média, a menor e a maior das
idades\n");

        scanf("%d", &opcao);
        switch (opcao)
        {
        case 0:
        {
            printf("PROGRAMA ENCERRADO!\n");
            break;
        }
        case 1:
        {
            FPVazia(&Pilha);
            break;
        }
        case 2:
        {
            resposta = Vazia(Pilha);
            if (resposta == 1)
                printf("Pilha está vazia");
            else
                printf("Pilha não está vazia");
```

```
            break;
        }
        case 3:
        {
            printf("Digite o código: ");
            scanf("%d", &x.cod);
            printf("Digite a idade: ");
            scanf("%d", &x.idade);
            Empilhar(&Pilha, &x);
            break;
        }
        case 4:
        {
            Desempilhar(&Pilha, &x);
            break;
        }
        case 5:
        {
            ImprimeFundoTopo(Pilha);
            break;
        }
        case 6:
        {
            ImprimeTopoFundo(Pilha);
            break;
        }
        case 7:
        {
            printf("O tamanho da pilha é %d", Tamanho(&Pilha));
            break;
        }
        case 8:
        {
            printf("Insira o código: ");
            scanf("%d", &x.cod);
            Procura(&Pilha, &x);
            break;
        }
        case 9:
        {
            Divide(&Pilha, &Pilha2, &Pilha3);
            break;
```

```
        }
        case 10:
        {
            CalculaIdade(&Pilha);
            break;
        }
        default:
            printf("\n\nOpção inválida\n\n");
        }
        _getch();
        system("cls");
    } while (opcao != 0);

    return 0;
}
```

EXERCÍCIO 02

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

#define MaxTam 6
typedef int Apontador;
typedef struct
{
    int valor;
} TipoItem;

typedef struct
{
    TipoItem Item[MaxTam];
    Apontador Topo;
} TipoPilha;

TipoPilha Pilha_Pares;
TipoPilha Pilha_Impares;
TipoItem x;

void FPVazia(TipoPilha *Pilha)
{
    Pilha->Topo = 0;
```

```c
}

void Empilhar(TipoPilha *Pilha, TipoItem *x)
{
    if (Pilha->Topo >= MaxTam)
    {
        if (Pilha->Item[Pilha->Topo - 1].valor % 2 == 0)
            printf("\nERRO: Pilha dos Pares está cheia\n");
        else
            printf("\nERRO: Pilha dos Ímpares está cheia\n");
    }
    else
    {
        Pilha->Item[Pilha->Topo] = *x;
        Pilha->Topo++;
    }
}

void ImprimeTopoFundo(TipoPilha Pilha)
{
    for (int i = Pilha.Topo - 1; i >= 0; i--)
    {
        printf("%d   ", Pilha.Item[i].valor);
    }
}

int main()
{
    char resp;
    SetConsoleOutputCP(CP_UTF8);
    FPVazia(&Pilha_Pares);
    FPVazia(&Pilha_Impares);
    for (int i = 0; i < 12; i++)
    {
        printf("\nDigite um número: ");
        scanf("%d", &x.valor);
        if (x.valor % 2 == 0)
        {
            Empilhar(&Pilha_Pares, &x);
        }
        else
        {
```

```
            Empilhar(&Pilha_Impares, &x);
        }
        printf("\nMais um número? [S/N]: ");
        scanf("%s", &resp);
        if (resp == 'N' || resp == 'n' || (Pilha_Pares.Topo >= MaxTam
&& Pilha_Impares.Topo >= MaxTam))
            break;
    }
    printf("\n\nPilha dos Pares:\n");
    ImprimeTopoFundo(Pilha_Pares);
    printf("\n\nPilha dos Impares:\n");
    ImprimeTopoFundo(Pilha_Impares);

    return 0;
}
```